Experiment 2

Implement client-server communication using socket programming and TCP as transport layer protocol

Server program

Step1: Create a socket by calling socket() function

Step 2: Bind socket to server IP address and port number using bind() function

Step 3: Listen for incoming connection requests using listen() function

Step 4: Accept the connection requests using accept() function

Step5 : Receive data from client using recv() function

Step 6: Close the socket using close() function

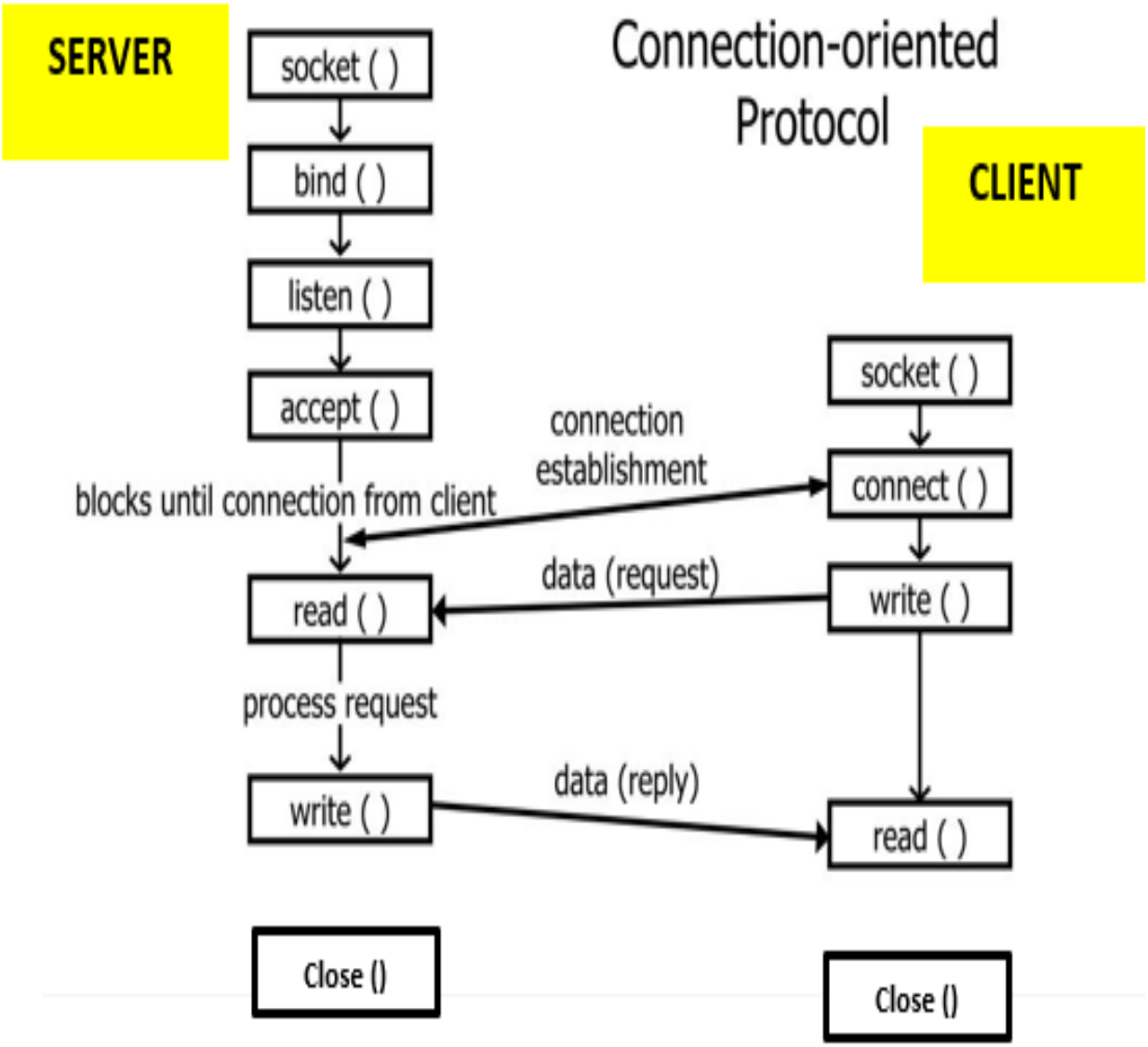
Client program

Step1: Create a socket by calling socket() function

Step 2: Connect to the server by calling the connect() function

Step 3: Send data to server using send() function

Step 4: Close the socket using close() function


SERVER
socket ( )
bind ( )
listen ( )
accept ( )
blocks until connection from client
read ( )
process request
write ( )
Close ()
Connection-oriented Protocol
CLIENT
socket ( )
connection establishment
connect ( )
data (request)
write ( )
data (reply)
read ( )
Close ()

A **socket** in computer networking refers to an endpoint for communication between two machines or processes over a network. It allows programs to send and receive data through the network. Sockets are a crucial part of network programming, especially in client-server applications.

A communication endpoint consists of:

An **IP address**: Identifies the host on the network.

A **port number**: Identifies the specific service or application running on the host.

Endpoint = (192.168.1.1, Port 8080)

Server program

Step 1: Creates a TCP socket that uses IPv4.

sock_desc = socket(AF_INET, SOCK_STREAM, 0);

- 1st argument of socket function call specifies the **address family**.
- Use the constant AF_INET as the first argument to specify IPv4 address family. AF_INET represents IPv4 address family
- Use the constant AF_INET6 as the first argument to specify IPv6 address family. AF_INET6 represents IPv6 address family
- 2nd argument of socket() specifies the **type of socket**:
- SOCK_STREAM: For connection-oriented communication
- SOCK_DGRAM: connectionless service
- 3rd argument specify protocol field.
- Pass 0 as 3rd argument to socket function. The OS will automatically select the correct protocol (TCP or UDP) based on address family and socket type. For eg, if we use AF_INET as 1st argument, SOCK_STREAM as 2nd argument and 3rd argument as 0, the OS will automatically choose TCP as the protocol
- socket() creates a socket and returns a socket descriptor, an integer value which will be stored in **sock_desc**

Step 2: The bind() function links a socket to a specific address (IP) and port.
The bind() function is used to associate a socket with a specific **IP address** and **port number.** Once bind() succeeds, the socket is tied to the specified address and port. For server applications, binding ensures that the application listens on a well-defined IP address and port.

socket(), bind(), listen(), accept() functions resides in **<sys/socket.h>**

The **struct sockaddr_in** resides in the **<netinet/in.h>** header file.

```
struct sockaddr_in
 {
short sin_family; // Address family (e.g., AF_INET for IPv4)
unsigned short sin_port; // Port number (in network byte order)
struct in_addr sin_addr; // IP address
char sin_zero[8]; // Padding (unused)
 };
```

```
struct sockaddr_in server_addr;
```

//server_addr is a variable of type **struct sockaddr_in**

//Setting IP address, port number and address family

```
server_addr.sin_family = AF_INET;
server_addr.sin_port = 2005;
server_addr.sin_addr.s_addr= INADDR_ANY;
```

INADDR_ANY makes the **server** listen on all network interfaces. When a server binds a socket to INADDR_ANY, the server listens for incoming connections on all network interfaces

If the server has multiple network interfaces (e.g., Ethernet, Wi-Fi), the server listens on all of them. . For example, if a server has multiple network interfaces with IPs 192.168.1.100 (Wi-Fi) and 10.0.0.2 (Ethernet), the server will accept connections on both.

The server can be configured to **bind** to a specific interface or listen on all interfaces. If the server binds to a specific **IP address** associated with one of its interfaces, the server will only accept connections that are directed to that particular IP address.

Invoke bind() function

```
bind(sock_desc, (struct sockaddr*)&server_addr,  sizeof(server_addr));
```

1st argument is the socket descriptor created using the socket() function**.**

2nd argument is the address of server_addr. This is a type cast that converts the pointer type from struct sockaddr_in* to struct sockaddr*. Converts the address from struct sockaddr_in to struct sockaddr so that bind() can accept it.

3rd argument is sizeof(server_addr)

Step 3: Listen for incoming connections using listen().

```
listen(sock_desc, 3);
```

2nd argument specifies the maximum number of queued unaccepted connections

At most, 3 connection requests will be queued.

A 4th client trying to connect will receive a "connection refused" error until one of the queued connections is handled.

If more clients attempt to connect than the queue can handle:

- The connection requests are either refused or ignored until space in the queue is available.

Step 4: Accept connections using accept().

Workflow of accept()

The accept() function:

- Removes the first pending connection from the queue.
- Populates the client_addr structure with the client's IP address and port no
- Creates a new socket to handle communication with that specific client and returns the new socket descriptor (client_sock_desc).

```
struct sockaddr_in client_addr;

socklen_t client_len;

client_len = sizeof(client_addr);

client_sock_desc = accept(sock_desc, (struct sockaddr*)&client_addr,
&client_len);

//receive data from the client

int bytes_received =recv(client_sock_desc, buf, 100, 0);
```

When the **flags** field(4th argument) in the recv() function is set to **0**, it means **default behavior**. In this case, recv() will simply wait for data to arrive on the socket, read it into the buffer, and remove it from the socket's queue.

buf: hold the incoming messages

printf("Message from client: %s\n", buf);

client program

step 1: Creates a TCP socket that uses IPv4.

sock_desc = socket(AF_INET, SOCK_STREAM, 0);

step 2: Connect to the server by calling the connect() function

struct sockaddr_in server;

server.sin_family=AF_INET;

server.sin_port=2005;

server.sin_addr.s_addr=inet_addr("127.0.0.1");

// inet_addr() converts this string into a binary representation that the system can use for communication over the network.

k=connect(sock_desc, (struct sockaddr*)&server, sizeof(server));

step 3: Send the data to the server using send() function

```
fgets(buf, 100, stdin);
k = send(sock_desc, buf, 100, 0);
```

'0' means that the function will wait (block) until all the data has been successfully sent

……UDP……..

sock_desc = socket(AF_INET, SOCK_DGRAM, 0);

recvfrom(sock_desc, buf, 100, 0, (struct sockaddr*)&client, sizeof(client));

sendto(sock_desc, buf, 100, 0, (struct sockaddr*)&client, sizeof(client));

UDP Server

socket(), bind(), recvfrom() sendto(),

UDP Client

socket(), fgets(buf,100,0) sendto(), recvfrom()